

MFR Virtual Lab — Project Status Report

Steady-State Mixed Flow Reactor simulator · Auth + Roles + Backend integration

Generated 22 May 2026 · Owner: Pranav Hathwar · Repo: chemical-vlab (branch `pranav`)

1. Executive summary

The existing Flutter MFR lab app has been extended with full authentication, role-based access (admin / student), a Node.js + Express + Prisma + PostgreSQL backend, and data persistence — **without modifying any of the original math/solver code**. The stack is running locally and verified: the backend passed a **22/22** end-to-end smoke test, `flutter analyze` is clean, and the web build compiles. Email/password and Google sign-in are wired to a live PostgreSQL database.

Current run state: Backend **RUNNING** at `http://localhost:4000` · Web app **RUNNING** at `http://localhost:8087` · PostgreSQL 18 database `mfr_lab` **ONLINE**

2. What was built / changed

Area	Delivered
Authentication	Email/password (bcrypt, 12 rounds); Google OAuth (live); GitHub OAuth (code complete, needs GitHub app)
Authorization	Role-based (admin vs student); admins set via <code>ADMIN_EMAILS</code> ; students get 403 on admin routes
Backend API	Express server: auth, sessions, trials, admin endpoints (see §6)
Persistence	Sessions & trials saved to PostgreSQL; resume-active-session supported
Hidden k security	Generated server-side; stored AES-256-GCM encrypted; key = HMAC-SHA256(studentId+sessionId, secret); decrypted only for admins
JWT	15-min access + 7-day refresh; refresh rotated on use & SHA-256 hashed at rest; silent auto-refresh via Dio interceptor
Reactor diagram	Replaced CustomPainter with <code>Image.asset('assets/images/reactor_diagram.png')</code> (placeholder PNG) + painter fallback via <code>errorBuilder</code>
Trial cap	Max trials 12 → 10 , enforced client AND server side
Student view	Graph hidden from students; admin sees graph + decrypted actual k + Excel export
Deployment config	Android (INTERNET, app label, mfrlab deep link, multiDex, SHA-1 note); backend Procfile, README, <code>.env.example</code> ; toolchain upgrade
Constants	Added <code>kMaxTrials</code> , <code>kMinK</code> , <code>kMaxK</code> , <code>kAppName</code> , <code>kAppVersion</code> , <code>kDeepLinkScheme</code> , <code>kReactorImagePath</code>

3. Tech stack & versions

Tooling

New Flutter packages

- `dio` ^5.4.3 — HTTP + interceptors

Component	Version
Flutter / Dart	3.41.6 / 3.11.4
Node / npm	v22.14.0 / 10.9.2
PostgreSQL	18
Prisma / Express	5.22 / 4.21
Kotlin / AGP / Gradle	2.1.0 / 8.9.1 / 8.11.1
compileSdk / targetSdk	36 / 34

- `flutter_secure_storage` ^9.2.2 — token storage
- `google_sign_in` ^6.2.1 — Google OAuth
- `flutter_web_auth_2` ^4.0.0 — GitHub OAuth
- `jwt_decoder` ^2.0.1 — token validity

Backend deps

- `express`, `@prisma/client`, `prisma`
- `bcryptjs`, `jsonwebtoken`, `helmet`
- `cors`, `express-rate-limit`, `express-validator`, `dotenv`

4. Tests run & results

Test	Result	Detail
Backend end-to-end smoke test	22 / 22 PASS	scripts/smoke-test.js against live PostgreSQL
<code>flutter analyze</code>	CLEAN	0 errors, 0 warnings (incl. excel_exporter)
<code>flutter build web</code>	SUCCESS	compiles; reactor image asset bundled & served (HTTP 200)
Backend module load + crypto	PASS	AES-256 encrypt/decrypt round-trip; $k \in [0.25, 0.50]$
Prisma migration	IN SYNC	4 tables created (users, refresh_tokens, sessions, trials)
Live health checks	OK	app :8087 → 200; backend :4000 → ok
Release APK build	IN PROGRESS	toolchain blocker fixed; build paused to run web (see §8)
Google OAuth E2E	NEAR COMPLETE	client ID wired, People API enabled, access-token flow live; final user confirmation pending
GitHub OAuth E2E	NOT CONFIGURED	code complete; needs a GitHub OAuth app

Smoke test — individual checks (all PASS)

health · register student (role=student) · login · /me · create session · hiddenK in [0.25,0.50] · active-session resume · NOT k-revealed early · record 3 trials · submit k · server actualK == issued k · accuracy % correct · student blocked from /api/admin/* (403) · 11th trial rejected (server) · refresh rotation · admin role via ADMIN_EMAILS · admin /stats · admin /students · admin sees DECRYPTED k · encrypted_k stored as iv.tag.cipher (not raw)

5. Security audit (all clear)

Check	Result
<code>double.parse(</code> in lib/	0 (uses tryParse)
MFRGraph in student screens	0 usages (admin only)

<code>hiddenK</code> exposure	only provider + API transport, never a screen/widget
<code>password_hash</code> in responses	never returned (publicUser strips it)
<code>encrypted_k</code> in student responses	never returned
helmet / CORS allow-list / rate limiting	applied (5/min auth, 100/min api)
Prisma client singleton	yes
<code>print()</code> / <code>console.log</code> of secrets	none

6. API endpoints

Method	Path	Access
GET	<code>/health</code>	public
POST	<code>/api/auth/register</code> · <code>/login</code> · <code>/refresh</code> · <code>/logout</code>	public
GET	<code>/api/auth/me</code>	any auth
POST	<code>/api/auth/google</code> · <code>/github</code>	public (OAuth)
POST	<code>/api/sessions</code> · <code>/:id/trials</code> · <code>/:id/submit</code>	student
GET	<code>/api/sessions/active</code>	student
GET	<code>/api/admin/stats</code> · <code>/students</code> · <code>/students/:id</code> · <code>/sessions/:id</code>	admin

7. Access details

Demo logins

Role	Email	Password
Admin	admin@demo.com	demo1234
Student	student@demo.com	demo1234
Admin (alt)	professor@university.edu	secret123

Google OAuth

- Project: `swaathi-portfolio` (885363054408)
- Client ID: `885363054408-83umtp17...apps.googleusercontent.com`
- People API: **enabled**
- Authorized JS origin: `http://localhost:8087`
- Flow: access-token (web) → backend tokeninfo+userinfo

Local DB: `postgresql://postgres:postgres123@localhost:5432/mfr_lab` (secrets live in `backend/.env`, which is git-ignored).

8. Pending / next steps

- **Confirm Google sign-in end-to-end** — relaunched with the access-token fix; verify it lands in the app.
- **Finish release APK build** — Kotlin/AGP/Gradle/compileSdk fixed; resume `flutter build apk --release` when not running the web app (build locks conflict).
- **GitHub OAuth** — create a GitHub OAuth app; set client id/secret + callback in `backend/.env`; pass `--dart-define=GITHUB_CLIENT_ID`.

- Deployment — host Flutter web + backend publicly; add the public origin to Google; update `CORS_ORIGINS` and `API_BASE_URL` .
- Pre-release — strong production JWT/encryption secrets; set real `ADMIN_EMAILS` ; add release keystore SHA-1 to Google Console; drop the real reactor image into `assets/images/reactor_diagram.png` .

9. How to resume

1) PostgreSQL — ensure the `postgresql-x64-18` service is running.
2) Backend — `cd backend && npm run dev` (→ `http://localhost:4000`)
3) Web app (Google enabled) —
`cd mfr_lab && flutter run -d chrome --web-port=8087 --dart-define=GOOGLE_CLIENT_ID=885363054408-83umtp17dlgaql9732958ecchce8nf9p.apps.googleusercontent.com`
4) Backend smoke test — `node scripts/smoke-test.js` (server must be running)
Docs — see `INTEGRATION_README.md` (root) and `backend/README.md` .

10. Key files (created this engagement)

Backend (backend/)

- `src/index.js`, `prisma.js`
- `config/jwt.js`, `config/encryption.js`
- `middleware/auth`, `role`, `validate`, `rateLimit`
- `routes/auth`, `session`, `admin`
- `controllers/auth`, `session`, `admin`
- `services/auth`, `session`, `encryption`, `oauth`
- `prisma/schema.prisma`, `scripts/{smoke-test,promote-admin}.js`
- `package.json`, `Procfile`, `README.md`, `.env(example)`

Flutter (mfr_lab/lib/)

- `constants.dart`
- `models/{user,session,api_response}_model.dart`
- `providers/auth_provider.dart`
- `services/{api,auth,token,session}_service.dart`
- `screens/auth/{login,register,forgot_password}_screen.dart`
- `screens/admin/{admin_dashboard,student_list,student_detail}_screen.dart`
- `widgets/auth/social_login_button.dart`
- `widgets/admin/{student,session,stats}_card.dart`
- `main.dart` (`AuthWrapper`), `home_screen.dart`, `experiment_provider.dart` (modified)

Untouched (per rules): `mfr_solver.dart`, `trial_model.dart`, `mfr_graph.dart`, `reactor_diagram.dart`, `trial_table.dart`, `input_field.dart`.
(`excel_exporter.dart` had a 2-line null-safety cleanup only, to keep analyze clean.)